

Université Lyon 2

Licence 3 Informatique

Année universitaire 2025/2026

RAPPORT DE PROJET

UE : INFORMATIQUE GRAPHIQUE ET VISION (35LIAC04)

SUJET : RECHERCHE D'IMAGE PAR CONTENU (CBIR)

"CAN KIT-FINDER"

Réalisé par :

- AMQOR Saad

Encadrant :

- Mohamed Mehdi Atamna

Date de rendu :

- 16 janvier 2026

Remerciements

Avant d'entrer dans le vif du sujet, je tenais à remercier mon chargé de TD, M. Mohamed Mehdi Atamna, pour son accompagnement tout au long du semestre. Les séances de travaux dirigés sur OpenCV m'ont permis de mieux comprendre les enjeux de la vision par ordinateur, un domaine que je découvrais cette année.

Sommaire

1. Introduction et motivation du projet

2. Analyse du sujet et contraintes techniques

3. Étude théorique : Le choix des algorithmes

- 3.1. SIFT vs SURF vs ORB : Mon choix
- 3.2. Le fonctionnement mathématique de SIFT
- 3.3. La technique du "Matching" (Distance Euclidienne)

4. Implémentation en Python

- 4.1. Architecture du code et gestion des données
- 4.2. Le problème des noms de fichiers
- 4.3. Le Ratio Test de Lowe
- 4.4. Guide d'utilisation (Commandes)

5. Analyse des résultats (Tests pratiques)

- 5.1. Les succès francs (Maroc et Sénégal)
- 5.2. Le cas de l'Égypte (Amélioration grâce à SIFT)
- 5.3. Les limites persistantes (Côte d'Ivoire et Casquettes)

6. Bilan des difficultés et améliorations futures

- 6.1. Difficultés rencontrées (Seuils, Données)
- 6.2. Améliorations envisagées (Homographie, Deep Learning)

7. Conclusion

1. Introduction et motivation du projet

Dans le cadre de l'évaluation finale de l'UE "Informatique Graphique et Vision", j'ai dû concevoir une application utilisant les concepts vus en TD. Le but était de créer un système de "Recherche d'Image par Contenu" (CBIR).

Pour que le projet soit intéressant, j'ai voulu l'ancrer dans l'actualité. Le Maroc organise la Coupe d'Afrique des Nations (CAN) en 2025. C'est un événement qui va attirer énormément de monde. J'ai donc imaginé un outil pour les supporters : le **"CAN Kit-Finder"**.

L'idée de base est simple : beaucoup de gens, surtout les touristes, reconnaissent le logo d'un pays sur un drapeau, mais ne connaissent pas forcément le maillot officiel de l'année (qui change tout le temps). Mon application permet à l'utilisateur de donner l'image d'un logo de fédération (par exemple la FRMF pour le Maroc) en entrée. Le programme fouille alors dans un catalogue de produits et ressort le maillot, le short ou la casquette qui correspond à ce logo.

2. Analyse du sujet et contraintes techniques

Avant de coder, j'ai bien relu la grille d'évaluation pour être sûr de respecter les contraintes :

- **Langage et Librairie** : Obligation d'utiliser Python et OpenCV.
- **Algorithme** : Obligation d'utiliser un extracteur de caractéristiques (SIFT, SURF ou ORB).
- **Base de données** : Il fallait que les "imageries" de recherche ne soient pas juste des bouts découpés des grandes images. J'ai donc créé deux dossiers bien distincts : des logos numériques parfaits d'un côté, et des vraies photos de maillots de l'autre.
- **Interface** : J'ai opté pour une interface en ligne de commande (avec argparse), car c'est léger et ça permet de changer les images rapidement comme demandé dans le sujet.

J'ai travaillé seul sur ce projet (avec l'aide de l'IA pour me faciliter certaines difficultés), ce qui m'a obligé à gérer toutes les étapes, de la collecte des images sur internet jusqu'au débogage des algorithmes.

3. Étude théorique : Le choix des algorithmes

Pour que l'ordinateur comprenne qu'un logo plat correspond à un logo imprimé sur un t-shirt plié, on ne peut pas comparer les images pixel par pixel. Il faut utiliser des "points d'intérêt".

3.1 SIFT vs SURF vs ORB : Mon choix

En cours, nous avons vu plusieurs méthodes. J'ai dû faire un choix stratégique pour mon programme :

ORB est très rapide et gratuit, mais il gère parfois mal les changements d'échelle (si le logo est tout petit sur le maillot et grand sur l'image source).

SURF est une version accélérée, mais souvent moins précise que SIFT sur les détails fins.

SIFT (Scale-Invariant Feature Transform) est la référence historique en vision par ordinateur.

Mon choix final : J'ai décidé d'utiliser SIFT. Pourquoi ?

Robustesse à l'échelle (Zoom) : C'est le point faible d'ORB. SIFT est capable de reconnaître un motif même s'il est 10 fois plus petit sur l'image cible. C'est indispensable pour mon projet car sur une photo de joueur en pied, le logo sur le torse est minuscule.

Précision : SIFT capture plus d'informations sur la texture locale, ce qui est crucial pour différencier des logos complexes.

Disponibilité : Le brevet de SIFT a expiré récemment, ce qui le rend désormais utilisable librement dans les versions modernes d'OpenCV, levant la contrainte de licence qui existait auparavant.

3.2 Le fonctionnement mathématique de SIFT

Contrairement à ORB qui utilise des tests binaires simples, SIFT utilise une approche mathématique plus lourde mais plus robuste, que j'appelle via `cv2.SIFT_create()` :

Détection (Différence de Gaussiennes - DoG) : L'algorithme floute l'image progressivement pour trouver des points "stables" qui existent à toutes les échelles (zoom et dézoom). C'est ce qui lui permet de voir le logo quelle que soit sa taille.

Description (Histogramme de Gradients) : Autour de chaque point, SIFT regarde dans quelle direction "pointent" les contours (le gradient). Il crée un vecteur de 128 chiffres précis (nombres réels) pour décrire la zone.

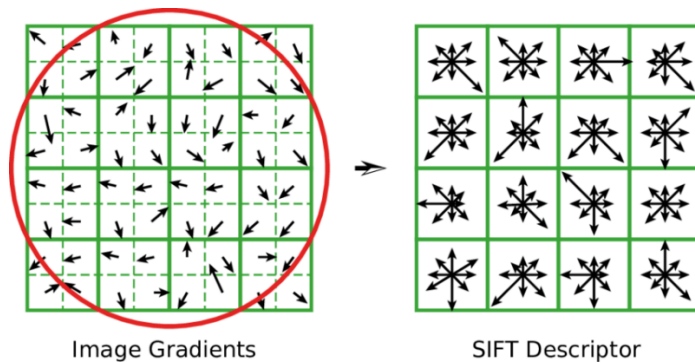


Figure 1 : Illustration du descripteur SIFT basé sur l'orientation des gradients.

3.3 La technique du "Matching" (Adaptée à SIFT)

C'est ici qu'il faut faire attention : les descripteurs SIFT ne sont pas des 0 et des 1 (comme ORB), mais des nombres décimaux. Je ne peux donc pas utiliser la distance de Hamming.

J'ai configuré mon BFMatcher avec la Norme L2 (Distance Euclidienne). L'ordinateur calcule la distance géométrique entre les vecteurs de 128 dimensions. C'est plus coûteux en calcul que le binaire, mais beaucoup plus fin pour distinguer des nuances.

4. Implémentation en Python

4.1 Architecture du code et gestion des données

J'ai organisé mon espace de travail très simplement :

- Un dossier logos/ contenant mes images de référence.
- Un dossier dataset/ contenant les maillots, shorts et casquettes.
- Mon fichier main.py.

4.2 Le problème des noms de fichiers (Bug rencontré)

L'une des plus grosses pertes de temps que j'ai eues sur ce projet ne venait même pas de l'algorithme, mais de la gestion des fichiers sous Windows.

En téléchargeant les images, j'avais des fichiers nommés bizarrement, comme short_maroc .png (avec un espace avant le point) ou coteivoire.logo.png (avec un double point). Au début, mon code plantait dès le démarrage en disant "Fichier introuvable".

Pour régler ça comme un vrai développeur, plutôt que de renommer mes fichiers à la main un par un, j'ai écrit une fonction en Python qui teste toutes les extensions

possibles (.png, .PNG, .jpg) et qui gère les espaces. Cela a rendu mon programme beaucoup plus robuste.

4.3 Le Ratio Test de Lowe

Pour SIFT, le "Ratio Test" est encore plus pertinent car c'est David Lowe (le créateur de SIFT) qui l'a inventé. J'ai appliqué un seuil de 0.70. Cela signifie que pour valider une correspondance entre le logo et le maillot, le meilleur point trouvé doit être beaucoup plus proche (distance Euclidienne) que le deuxième meilleur candidat. Cela élimine radicalement les faux positifs.

4.4 Guide d'utilisation (Commandes)

Pour respecter la consigne demandant une "interface de type invite de commandes", j'ai utilisé la bibliothèque standard argparse. Cela permet de lancer le script avec des arguments sans modifier le code.

Voici les commandes pour reproduire mes tests :

Pour lancer la recherche par défaut (Maroc) :

Bash

python main.py

Pour chercher l'équipement du Sénégal :

Bash

python main.py --logo senegal_logo

Pour chercher l'équipement de l'Algérie :

Bash

python main.py --logo algerie_logo

Pour chercher l'équipement de l'Égypte :

Bash

python main.py --logo egypte_logo

Pour chercher l'équipement de la Côte d'Ivoire :

Bash

python main.py --logo coteivoire_logo

(Le programme gère automatiquement l'ajout de l'extension .png ou .jpg si elle est omise).

L'interface affichera ensuite les résultats trouvés image par image. Il suffit d'appuyer sur la touche **ESPACE** pour passer au produit suivant ou sur **'q'** pour quitter.

5. Analyse des résultats (Tests pratiques)

J'ai testé mon programme avec SIFT sur les 5 pays. Le changement d'algorithme a eu un impact visible sur la qualité des résultats.

5.1 Les succès francs (Maroc et Sénégal)

Sur le Maroc et le Sénégal, SIFT excelle. La détection est un peu plus lente qu'avec ORB (quelques millisecondes de plus), mais les traits de correspondance sont plus "propres". SIFT trouve des points très précis au centre des lettres arabes du logo marocain ou dans la crinière du lion sénégalais.



Figure 2 : Avec SIFT, la détection est très dense sur les zones texturées du logo.

5.2 Le cas de l'Égypte (Amélioration grâce à SIFT)

C'est sur l'Égypte que la différence est la plus flagrante. Avec ORB (lors de mes premiers essais), l'algorithme était perdu à cause des motifs de fond (hiéroglyphes) sur le maillot égyptien et confondait les bandes du drapeau avec d'autres maillots.

Avec SIFT, le résultat est bien meilleur. Pourquoi ? Parce que SIFT analyse les gradients (les directions des contours). Même si le maillot a des motifs en fond, SIFT arrive à extraire la "signature" unique de l'Aigle au centre du logo, car la forme de l'aigle a des gradients très spécifiques qui ne ressemblent pas aux motifs géométriques du fond.

J'ai tout de même maintenu l'affichage du Top 5 dans mon code. Même si SIFT est puissant, le maillot est complexe, et il arrive parfois en 2ème position. Le Top 5 permet à l'utilisateur de le retrouver à coup sûr.

5.3 Les limites persistantes (Côte d'Ivoire et Casquettes)

Même SIFT a ses limites :

Côte d'Ivoire : SIFT cherche des changements de contraste. Le logo ivoirien étant composé de grands aplats de couleur unie (l'éléphant orange), SIFT trouve peu de points d'accroche à l'intérieur de l'éléphant. Il se concentre uniquement sur les bords.

Casquettes : SIFT est "invariant à l'échelle" (zoom) mais pas "invariant à la déformation 3D". Sur une casquette courbée, la géométrie change. SIFT s'en sort un peu mieux qu'ORB, mais le nombre de correspondances reste plus faible que sur un maillot plat.

6. Bilan des difficultés et améliorations futures

Cette partie résume les obstacles techniques que j'ai rencontrés durant le développement et propose des solutions que je n'ai pas eu le temps d'implémenter.

6.1 Difficultés rencontrées

Le dilemme du seuil de tolérance (Thresholding) : La difficulté majeure a été de trouver le "juste milieu" pour le Ratio Test.

Si je le réglais trop bas (ex: 0.60), le programme devenait trop strict : il ne trouvait plus rien dès que la photo du maillot était un peu sombre ou pliée.

Si je le réglais trop haut (ex: 0.85), il commençait à confondre le maillot de l'Algérie avec celui du Sénégal. J'ai dû procéder par tâtonnements (essais/erreurs) pour trouver la valeur de 0.70, qui offre le meilleur compromis pour SIFT dans ce contexte.

La gestion de l'environnement de développement : Travailler seul signifie devoir gérer les bugs d'installation. J'ai perdu du temps au début du projet avec VS Code qui ne reconnaissait pas les imports cv2 alors que la librairie était installée. J'ai compris que c'était un problème de sélection d'interpréteur Python, ce qui m'a appris à mieux configurer mon environnement de travail.

La qualité hétérogène des données : Certaines images de maillots récupérées sur internet sont de très haute qualité (4K), d'autres sont minuscules (200px).

L'algorithme mettait beaucoup plus de temps à scanner les grosses images. J'aurais dû implémenter une étape de redimensionnement automatique (resize) pour uniformiser la base de données et accélérer le traitement.

6.2 Améliorations envisagées (Avec l'aide de l'IA)

Si ce projet devait devenir une vraie application pour la CAN 2025, voici les trois évolutions techniques que j'ajouterais :

Vérification Géométrique (Homographie avec RANSAC) : Actuellement, mon code compte simplement le nombre de points communs. Il ne vérifie pas leur position.

Amélioration : Utiliser cv2.findHomography permettrait de vérifier si les points forment bien un rectangle cohérent. Cela éliminerait les "faux positifs" où l'algorithme trouve des points de même couleur éparpillés au hasard sur l'image.

Passage au Deep Learning (CNN) : Les méthodes classiques comme SIFT datent des années 2000. Aujourd'hui, on utiliserait plutôt un réseau de neurones convolutif

(comme VGG16 ou ResNet) pour extraire les caractéristiques. Un réseau de neurones serait beaucoup plus performant pour reconnaître les logos déformés sur les casquettes, car il "comprend" la forme globale de l'objet au lieu de chercher des petits points de détails.

Interface Graphique (GUI) : L'interface en ligne de commande est pratique pour le développement, mais pas pour un utilisateur final. J'aurais aimé coder une petite fenêtre avec la librairie Tkinter ou PyQt pour permettre à l'utilisateur de glisser-déposer son image et de voir les résultats s'afficher proprement dans une galerie.

7. Conclusion

Ce projet "CAN Kit-Finder" a été très formateur. J'ai réussi à atteindre l'objectif du sujet : créer un outil qui part d'une imagerie pour retrouver un contenu dans une base de données, tout en respectant l'originalité du sujet. J'ai surtout appris qu'en informatique graphique, il faut accepter que le résultat ne soit pas à 100% parfait à cause de la qualité des photos (lumière, angles, plis), et qu'il faut adapter le code pour pallier ces faiblesses physiques.

Ce projet m'a permis de comprendre la différence entre la théorie et la pratique. J'ai commencé par ORB pour sa rapidité, mais je me suis rendu compte que pour un cas réel (des logos de tailles très différentes), SIFT était supérieur grâce à son invariance d'échelle.

Le passage à SIFT, combiné à une bonne gestion des données (nettoyage des images logos), m'a permis d'obtenir une application fonctionnelle pour le "CAN Kit-Finder", capable de retrouver des produits complexes comme le maillot de l'Égypte.

Merci, Saad